

Analyzing E-Learning Platform Purchases using MySQL

MySQL Module Capstone Project - Project Document

1. Objective

An online learning platform sells digital courses to learners across different countries, where each learner can purchase multiple courses and each course belongs to a specific category. The objective of this project is to design, populate, and query a relational MySQL database that enables the management team to derive meaningful insights into:

- Sales trends across courses and categories
- Learner purchasing behavior and spending patterns
- Popular and underperforming course categories

This is achieved by building a three-table relational schema (learners, courses, purchases), populating it with representative sample data, and writing a structured set of SQL queries covering joins, aggregation, subqueries, correlated subqueries, common table expressions (CTEs), CASE-based classification, NULL handling, and a reusable database view.

2. Database Schema

The database `elearning_platform` consists of three related tables.

2.1 learners

Column	Type	Constraint	Description
<code>learner_id</code>	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each learner
<code>full_name</code>	VARCHAR(100)	NOT NULL	Full name of the learner
<code>country</code>	VARCHAR(50)	NOT NULL	Country of residence

2.2 courses

Column	Type	Constraint	Description
<code>course_id</code>	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each course
<code>course_name</code>	VARCHAR(100)	NOT NULL	Title of the course
<code>category</code>	VARCHAR(50)	NOT NULL	Category the course belongs to
<code>unit_price</code>	DECIMAL(10,2)	NOT NULL	Price per course

2.3 purchases

Column	Type	Constraint	Description
<code>purchase_id</code>	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each purchase transaction
<code>learner_id</code>	INT	FOREIGN KEY → learners	The learner who made the purchase
<code>course_id</code>	INT	FOREIGN KEY → courses	The course that was purchased
<code>quantity</code>	INT	NOT NULL	Number of course units purchased
<code>purchase_date</code>	DATE	NOT NULL	Date the purchase was made

2.4 Entity Relationship Summary

learners (1) <----> purchases >---- (1) courses

Each learner can have many purchases, and each course can appear in many purchases, but every purchase references exactly one learner and one course — a classic many-to-many relationship between learners and courses, resolved through the purchases junction table.

3. Constraints Implemented

The schema enforces referential integrity and data quality through the following constraints:

Constraint Type	Applied On	Purpose
PRIMARY KEY	learner_id, course_id, purchase_id	Uniquely identifies each row; auto-incremented for every insert.
FOREIGN KEY	purchases.learner_id → learners.learner_id	Ensures every purchase references a learner that actually exists.
FOREIGN KEY	purchases.course_id → courses.course_id	Ensures every purchase references a course that actually exists.
NOT NULL	full_name, country, course_name, category, unit_price, learner_id, course_id, quantity, purchase_date	Prevents missing values in fields required for every analytical query.
AUTO_INCREMENT	learner_id, course_id, purchase_id	Generates unique sequential IDs automatically on insert.
DECIMAL(10,2)	unit_price	Stores currency values precisely to 2 decimal places, avoiding floating-point rounding errors.

Together, these constraints ensure that no purchase can exist without a valid learner and course, that every row has a unique identifier, that no required field is left blank, and that monetary values are stored with exact precision rather than approximate floating-point values.

4. Step-by-Step Execution Log

The complete script (elearning_analysis.sql) was executed sequentially against a MySQL 8.0-compatible server. The table below documents each step in the order it was run, along with the verified outcome.

Step	Action Executed	Outcome / Result
1	DROP/CREATE DATABASE elearning_platform; CREATE TABLE learners, courses, purchases	Database and 3 tables created with primary keys, foreign keys, and NOT NULL constraints. No errors.
2	INSERT sample data into learners, courses, purchases	5 learners, 6 courses (5 categories), 8 purchase records inserted successfully.
3	INNER JOIN across purchases, learners, courses	8 rows returned, formatted currency, sorted by highest total amount. Top row: Kenji Sato – Full Stack Web Development – 11,998.00.
4	LEFT JOIN from learners to purchases/courses	All learners returned with their purchase details; confirms no learner is excluded even if they had zero purchases.
5	RIGHT JOIN from purchases to courses	All 6 courses returned, including UX Design Basics with NULL learner/quantity — proves the course was never purchased.
6	Q1 – Total spending per learner with country	5 rows returned; Kenji Sato highest at 16,997.00, Sophia Rossi lowest at 999.00.
7	Q2 – Top 3 courses by quantity sold	Python for Beginners (4 units), Advanced Data Science (3 units), Excel Fundamentals (3 units).
8	Q3 – Revenue and unique learners per category	Web Development highest revenue (17,997.00); Beginner category widest reach (3 unique learners).
9	Q4 – Learners purchasing from more than one category	3 of 5 learners qualified: Rahul Mehta, Kenji Sato, Alice Johnson.
10	Q5 – Courses never purchased	1 row returned: UX Design Basics (Design category).
11	Q6 – Learners spending above average (subquery)	Average spending computed as 8,877.00; 3 learners exceeded it: Kenji Sato, Rahul Mehta, Alice Johnson.
12	Q7 – Courses priced above any Beginner-category course (ANY subquery)	5 of 6 courses qualified, all except Excel Fundamentals (the lowest-priced Beginner course).

13	Q8 – Learners above their own country’s average spending (correlated subquery)	1 row returned: Alice Johnson (USA), the only learner exceeding her country’s average.
14	Q9 – CTE for total spending, filtered above 10,000	2 learners qualified: Kenji Sato (16,997.00), Rahul Mehta (14,996.00).
15	Q10 – CASE classification into High/Medium/Low Value	All 5 learners classified: 1 High Value, 2 Medium Value, 2 Low Value.
16	Q11 – NULL handling with IFNULL/COALESCE	All 6 courses listed; UX Design Basics shows purchase_count = 0 and total_quantity_sold = 0 instead of NULL.
17	Q12 – CREATE VIEW category_performance_view and query it	View created successfully; 4 categories returned with revenue, purchase count, and average revenue per purchase (Design excluded as it has no purchases to aggregate).

SECTION 1: DATABASE SETUP & DATA ENTRY

The screenshot shows the MySQL Workbench interface with the 'elearning_analysis' script open. The script contains the following SQL commands:

```

4
5  -- SECTION 1: DATABASE SETUP & DATA ENTRY
6
7 • DROP DATABASE IF EXISTS elearning_platform;
8 • CREATE DATABASE elearning_platform;
9 • USE elearning_platform;
10
11  -----

```

The Output pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	21:49:09	DROP DATABASE IF EXISTS elearning_platform	0 row(s) affected, 1 warning(s): 1008 Can't drop database 'elearning_platform'; database doesn't exist	0.063 sec
2	21:49:10	CREATE DATABASE elearning_platform	1 row(s) affected	0.032 sec
3	21:49:10	USE elearning_platform	0 row(s) affected	0.000 sec

The screenshot shows the MySQL Workbench interface with the 'elearning_analysis' script open. The script contains the following SQL commands:

```

10
11  -----
12  -- Table: learners
13  -----
14 • CREATE TABLE learners (
15     learner_id INT PRIMARY KEY AUTO_INCREMENT,
16     full_name VARCHAR(100) NOT NULL,
17     country VARCHAR(50) NOT NULL
18 );
19
20  -----

```

The Output pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	21:49:09	DROP DATABASE IF EXISTS elearning_platform	0 row(s) affected, 1 warning(s): 1008 Can't drop database 'elearning_platform'; database doesn't exist	0.063 sec
2	21:49:10	CREATE DATABASE elearning_platform	1 row(s) affected	0.032 sec
3	21:49:10	USE elearning_platform	0 row(s) affected	0.000 sec
4	21:52:43	CREATE TABLE learners (learner_id INT PRIMARY KEY AUTO_INCREMENT, full_name...	0 row(s) affected	0.219 sec

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator: elearning_analysis

SCHEMAS

Filter objects

- btmpty
- btmpty1
- chatbot_db
- employee
- flask_practice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- practice
- Tables
- Views
- Stored Procedures
- Functions

Administration Schemas

Information

No object selected

```

19
20  -----
21  -- Table: courses
22  -----
23  CREATE TABLE courses (
24      course_id INT PRIMARY KEY AUTO_INCREMENT,
25      course_name VARCHAR(100) NOT NULL,
26      category VARCHAR(50) NOT NULL,
27      unit_price DECIMAL(10,2) NOT NULL
28  );
29

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	21:49:09	DROP DATABASE IF EXISTS elearning_platform	0 row(s) affected, 1 warning(s): 1008 Can't drop database 'elearning_platform'; database doesn't exist	0.063 sec
2	21:49:10	CREATE DATABASE elearning_platform	1 row(s) affected	0.032 sec
3	21:49:10	USE elearning_platform	0 row(s) affected	0.000 sec
4	21:52:43	CREATE TABLE learners (learner_id INT PRIMARY KEY AUTO_INCREMENT, full_name VARCHAR(100) NOT NULL, email VARCHAR(100) NOT NULL, password VARCHAR(100) NOT NULL, date_of_birth DATE NOT NULL, gender ENUM('M', 'F') NOT NULL);	0 row(s) affected	0.219 sec
5	21:53:15	CREATE TABLE courses (course_id INT PRIMARY KEY AUTO_INCREMENT, course_name VARCHAR(100) NOT NULL, category VARCHAR(50) NOT NULL, unit_price DECIMAL(10,2) NOT NULL);	0 row(s) affected	0.031 sec

Object Info Session

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator: elearning_analysis

SCHEMAS

Filter objects

- btmpty
- btmpty1
- chatbot_db
- employee
- flask_practice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- practice
- Tables
- Views
- Stored Procedures
- Functions

Administration Schemas

Information

No object selected

```

31  -----
32  -- Table: purchases
33  -----
34  CREATE TABLE purchases (
35      purchase_id INT PRIMARY KEY AUTO_INCREMENT,
36      learner_id INT NOT NULL,
37      course_id INT NOT NULL,
38      quantity INT NOT NULL,
39      purchase_date DATE NOT NULL,
40      CONSTRAINT fk_purchases_learner FOREIGN KEY (learner_id) REFERENCES learners(learner_id),
41      CONSTRAINT fk_purchases_course FOREIGN KEY (course_id) REFERENCES courses(course_id)
42  );
43  -----

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	21:49:09	DROP DATABASE IF EXISTS elearning_platform	0 row(s) affected, 1 warning(s): 1008 Can't drop database 'elearning_platform'; database doesn't exist	0.063 sec
2	21:49:10	CREATE DATABASE elearning_platform	1 row(s) affected	0.032 sec
3	21:49:10	USE elearning_platform	0 row(s) affected	0.000 sec
4	21:52:43	CREATE TABLE learners (learner_id INT PRIMARY KEY AUTO_INCREMENT, full_name VARCHAR(100) NOT NULL, email VARCHAR(100) NOT NULL, password VARCHAR(100) NOT NULL, date_of_birth DATE NOT NULL, gender ENUM('M', 'F') NOT NULL);	0 row(s) affected	0.219 sec
5	21:53:15	CREATE TABLE courses (course_id INT PRIMARY KEY AUTO_INCREMENT, course_name VARCHAR(100) NOT NULL, category VARCHAR(50) NOT NULL, unit_price DECIMAL(10,2) NOT NULL);	0 row(s) affected	0.031 sec
6	21:53:50	CREATE TABLE purchases (purchase_id INT PRIMARY KEY AUTO_INCREMENT, learner_id INT NOT NULL, course_id INT NOT NULL, quantity INT NOT NULL, purchase_date DATE NOT NULL, CONSTRAINT fk_purchases_learner FOREIGN KEY (learner_id) REFERENCES learners(learner_id), CONSTRAINT fk_purchases_course FOREIGN KEY (course_id) REFERENCES courses(course_id));	0 row(s) affected	0.172 sec

Object Info Session

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator: **elearning_analysis**

SCHEMAS

Filter objects

- btmpty
- btmpty1
- chatbot_db
- employee
- flask_pratice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- pratice
- Tables
- Views
- Stored Procedures
- Functions

Administration Schemas

Information

No object selected

```

43  -----
44  -- Sample Data: learners (5)
45  -----
46  • INSERT INTO learners (full_name, country) VALUES
47      ('Alice Johnson', 'USA'),
48      ('Rahul Mehta', 'India'),
49      ('Sophia Rossi', 'Italy'),
50      ('Kenji Sato', 'Japan'),
51      ('Maria Garcia', 'USA');
52
53  -----

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
2	21:49:10	CREATE DATABASE elearning_platform	1 row(s) affected	0.032 sec
3	21:49:10	USE elearning_platform	0 row(s) affected	0.000 sec
4	21:52:43	CREATE TABLE learners (learner_id INT PRIMARY KEY AUTO_INCREMENT, full_na...	0 row(s) affected	0.219 sec
5	21:53:15	CREATE TABLE courses (course_id INT PRIMARY KEY AUTO_INCREMENT, cours...	0 row(s) affected	0.031 sec
6	21:53:50	CREATE TABLE purchases (purchase_id INT PRIMARY KEY AUTO_INCREMENT, l...	0 row(s) affected	0.172 sec
7	21:54:41	INSERT INTO learners (full_name, country) VALUES ('Alice Johnson', 'USA), ('Rahul M...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.047 sec

Object Info Session

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator: **elearning_analysis**

SCHEMAS

Filter objects

- btmpty
- btmpty1
- chatbot_db
- employee
- flask_pratice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- pratice
- Tables
- Views
- Stored Procedures
- Functions

Administration Schemas

Information

No object selected

```

52
53  -----
54  -- Sample Data: courses (6, across 5 categories; one course is never purchased)
55  -----
56  • INSERT INTO courses (course_name, category, unit_price) VALUES
57      ('Python for Beginners', 'Beginner', 999.00),
58      ('Advanced Data Science', 'Data Science', 4999.00),
59      ('Digital Marketing Basics', 'Marketing', 2499.00),
60      ('Full Stack Web Development', 'Web Development', 5999.00),
61      ('Excel Fundamentals', 'Beginner', 799.00),
62      ('UX Design Basics', 'Design', 3499.00);

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
3	21:49:10	USE elearning_platform	0 row(s) affected	0.000 sec
4	21:52:43	CREATE TABLE learners (learner_id INT PRIMARY KEY AUTO_INCREMENT, full_na...	0 row(s) affected	0.219 sec
5	21:53:15	CREATE TABLE courses (course_id INT PRIMARY KEY AUTO_INCREMENT, cours...	0 row(s) affected	0.031 sec
6	21:53:50	CREATE TABLE purchases (purchase_id INT PRIMARY KEY AUTO_INCREMENT, l...	0 row(s) affected	0.172 sec
7	21:54:41	INSERT INTO learners (full_name, country) VALUES ('Alice Johnson', 'USA), ('Rahul M...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.047 sec
8	21:55:28	INSERT INTO courses (course_name, category, unit_price) VALUES ('Python for Beginner...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.031 sec

Object Info Session

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: elearning_analysis x

SCHMAS

Filter objects

- btmpty
- btmpty1
- chatbot_db
- employee
- flask_pratice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- praticed
- Tables
- Views
- Stored Procedures
- Functions

Administration Schemas

No object selected

Object Info Session

```
64
65 -- Sample Data: purchases (8)
66
67 INSERT INTO purchases (learner_id, course_id, quantity, purchase_date) VALUES
68 (1, 1, 3, '2026-01-05'), -- Alice - Python for Beginners
69 (1, 4, 1, '2026-02-10'), -- Alice - Full Stack Web Development
70 (2, 2, 2, '2026-01-15'), -- Rahul - Advanced Data Science
71 (2, 3, 2, '2026-03-01'), -- Rahul - Digital Marketing Basics
72 (3, 1, 1, '2026-01-20'), -- Sophia - Python for Beginners
73 (4, 4, 2, '2026-02-25'), -- Kenji - Full Stack Web Development
74 (4, 2, 1, '2026-03-10'), -- Kenji - Advanced Data Science
75 (5, 5, 3, '2026-01-30'), -- Maria - Excel Fundamentals
76 -- Note: course_id 6 (UX Design Basics) is intentionally never purchased
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
4	21:52:43	CREATE TABLE learners (learner_id INT PRIMARY KEY AUTO_INCREMENT, full_name VARCHAR(100) NOT NULL)	0 row(s) affected	0.219 sec
5	21:53:15	CREATE TABLE courses (course_id INT PRIMARY KEY AUTO_INCREMENT, course_name VARCHAR(100) NOT NULL)	0 row(s) affected	0.031 sec
6	21:53:50	CREATE TABLE purchases (purchase_id INT PRIMARY KEY AUTO_INCREMENT, learner_id INT NOT NULL, course_id INT NOT NULL, quantity INT, purchase_date DATE)	0 row(s) affected	0.172 sec
7	21:54:41	INSERT INTO learners (full_name, country) VALUES ('Alice Johnson', 'USA'), ('Rahul Mehta', 'India'), ('Sophia Rossi', 'Italy')	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.047 sec
8	21:55:28	INSERT INTO courses (course_name, category, unit_price) VALUES ('Python for Beginners', 'Beginner', 999.00), ('Full Stack Web Development', 'Intermediate', 1499.00), ('Advanced Data Science', 'Expert', 1999.00), ('Digital Marketing Basics', 'Beginner', 499.00), ('Excel Fundamentals', 'Beginner', 599.00), ('UX Design Basics', 'Beginner', 699.00)	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.031 sec
9	21:56:21	INSERT INTO purchases (learner_id, course_id, quantity, purchase_date) VALUES (1, 1, 3, '2026-01-05'), (1, 4, 1, '2026-02-10'), (2, 2, 2, '2026-01-15'), (2, 3, 2, '2026-03-01'), (3, 1, 1, '2026-01-20'), (4, 4, 2, '2026-02-25'), (4, 2, 1, '2026-03-10'), (5, 5, 3, '2026-01-30')	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	0.015 sec

SECTION 2: DATA EXPLORATION USING JOINS

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: elearning_analysis x

SCHMAS

Filter objects

- btmpty
- btmpty1
- chatbot_db
- employee
- flask_pratice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- praticed
- Tables
- Views
- Stored Procedures
- Functions

Administration Schemas

No object selected

Object Info Session

```
79
80 -- SECTION 2: DATA EXPLORATION USING JOINS
81
82
83 -- INNER JOIN: only purchases that have a matching learner and course
84 SELECT
85 l.full_name AS learner_name,
86 c.course_name AS course_name,
87 c.category AS category,
88 p.quantity AS quantity,
89 FORMAT(p.quantity * c.unit_price, 2) AS total_amount,
90 p.purchase_date AS purchase_date
91 FROM purchases p
92 INNER JOIN learners l ON p.learner_id = l.learner_id
93 INNER JOIN courses c ON p.course_id = c.course_id
94 ORDER BY (p.quantity * c.unit_price) DESC
```

Result Grid

learner_name	course_name	category	quantity	total_amount	purchase_date
Kenji Sato	Full Stack Web Development	Web Development	2	11,998.00	2026-02-25
Rahul Mehta	Advanced Data Science	Data Science	2	9,998.00	2026-01-15
Alice Johnson	Full Stack Web Development	Web Development	1	5,999.00	2026-02-10
Kenji Sato	Advanced Data Science	Data Science	1	4,999.00	2026-03-10
Rahul Mehta	Digital Marketing Basics	Marketing	2	4,998.00	2026-03-01

Result 1 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
8	21:55:28	INSERT INTO courses (course_name, category, unit_price) VALUES ('Python for Beginners', 'Beginner', 999.00), ('Full Stack Web Development', 'Intermediate', 1499.00), ('Advanced Data Science', 'Expert', 1999.00), ('Digital Marketing Basics', 'Beginner', 499.00), ('Excel Fundamentals', 'Beginner', 599.00), ('UX Design Basics', 'Beginner', 699.00)	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.031 sec
9	21:56:21	INSERT INTO purchases (learner_id, course_id, quantity, purchase_date) VALUES (1, 1, 3, '2026-01-05'), (1, 4, 1, '2026-02-10'), (2, 2, 2, '2026-01-15'), (2, 3, 2, '2026-03-01'), (3, 1, 1, '2026-01-20'), (4, 4, 2, '2026-02-25'), (4, 2, 1, '2026-03-10'), (5, 5, 3, '2026-01-30')	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	0.015 sec
10	21:57:10	SELECT l.full_name AS learner_name, c.course_name AS course_name, c.category AS category, p.quantity AS quantity, FORMAT(p.quantity * c.unit_price, 2) AS total_amount, p.purchase_date AS purchase_date FROM purchases p INNER JOIN learners l ON p.learner_id = l.learner_id INNER JOIN courses c ON p.course_id = c.course_id ORDER BY (p.quantity * c.unit_price) DESC	8 row(s) returned	0.078 sec / 0.000 sec

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

- btmpty
- btmpty1
- chatbot_db
- employee
- flask_practice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- practice
- Tables
- Views
- Stored Procedures
- Functions

Administration Schemas

Information

No object selected

Result Grid

Filter Rows: Limit to 1000 rows

Export: Wrap Cell Content: 15

Result 1

```

96 -- LEFT JOIN: all learners, including any with no purchases (none in this sample, but query is NULL-safe)
97 SELECT
98     l.full_name AS learner_name,
99     c.course_name AS course_name,
100     c.category AS category,
101     p.quantity AS quantity,
102     FORMAT(IFNULL(p.quantity * c.unit_price, 0), 2) AS total_amount,
103     p.purchase_date AS purchase_date
104 FROM learners l
105 LEFT JOIN purchases p ON l.learner_id = p.learner_id
106 LEFT JOIN courses c ON p.course_id = c.course_id
107 ORDER BY (p.quantity * c.unit_price) DESC;
108

```

learner_name	course_name	category	quantity	total_amount	purchase_date
Kenji Sato	Full Stack Web Development	Web Development	2	11,998.00	2026-02-25
Rahul Mehta	Advanced Data Science	Data Science	2	9,998.00	2026-01-15
Alice Johnson	Full Stack Web Development	Web Development	1	5,999.00	2026-02-10
Kenji Sato	Advanced Data Science	Data Science	1	4,999.00	2026-03-10
Rahul Mehta	Digital Marketing Basics	Marketing	2	4,998.00	2026-03-01

Result 2

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:00:38	SELECT l.full_name AS learner_name, c.course_name AS course_name	8 row(s) returned	0.016 sec / 0.000 sec

Object Info Session

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

- btmpty
- btmpty1
- chatbot_db
- employee
- flask_practice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- practice
- Tables
- Views
- Stored Procedures
- Functions

Administration Schemas

Information

No object selected

Result Grid

Filter Rows: Limit to 1000 rows

Export: Wrap Cell Content: 15

Result 3

```

109 -- RIGHT JOIN: all courses, including any never purchased (surfaces UX Design Basics)
110 SELECT
111     l.full_name AS learner_name,
112     c.course_name AS course_name,
113     c.category AS category,
114     p.quantity AS quantity,
115     FORMAT(IFNULL(p.quantity * c.unit_price, 0), 2) AS total_amount,
116     p.purchase_date AS purchase_date
117 FROM purchases p
118 RIGHT JOIN courses c ON p.course_id = c.course_id
119 LEFT JOIN learners l ON p.learner_id = l.learner_id
120 ORDER BY (p.quantity * c.unit_price) DESC;
121

```

learner_name	course_name	category	quantity	total_amount	purchase_date
Kenji Sato	Full Stack Web Development	Web Development	2	11,998.00	2026-02-25
Rahul Mehta	Advanced Data Science	Data Science	2	9,998.00	2026-01-15
Alice Johnson	Full Stack Web Development	Web Development	1	5,999.00	2026-02-10
Kenji Sato	Advanced Data Science	Data Science	1	4,999.00	2026-03-10
Rahul Mehta	Digital Marketing Basics	Marketing	2	4,998.00	2026-03-01

Result 3

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:00:38	SELECT l.full_name AS learner_name, c.course_name AS course_name	8 row(s) returned	0.016 sec / 0.000 sec
2	22:01:26	SELECT l.full_name AS learner_name, c.course_name AS course_name	9 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

SECTION 3: CORE ANALYTICAL QUERIES

Q1. Each learner's total spending with their country

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator: elearning_analysis

SCHEMAS

Filter objects

btmpty
btmpty1
chatbot_db
employee
flask_practice
form_data
leave_management_system
parkingmanagementsystem
plsql
practice
Tables
Views
Stored Procedures
Functions

Administration Schemas

No object selected

Information

Result Grid

Filter Rows: Limit to 1000 rows

Export: Wrap Cell Content: 1A

123 -- SECTION 3: CORE ANALYTICAL QUERIES
124
125 -- Q1. Each learner's total spending with their country
126 SELECT
127 1.full_name AS learner_name,
128 1.country AS country,
129 FORMAT(SUM(p.quantity * c.unit_price), 2) AS total_spending
130 FROM learners l
131 JOIN purchases p ON l.learner_id = p.learner_id
132 JOIN courses c ON p.course_id = c.course_id
133 GROUP BY 1.learner_id, 1.full_name, 1.country
134 ORDER BY SUM(p.quantity * c.unit_price) DESC;
135

Result 4

learner_name	country	total_spending
Kenji Sato	Japan	16,997.00
Rahul Mehta	India	14,996.00
Alice Johnson	USA	8,996.00
Maria Garcia	USA	2,397.00
Sophia Rossi	Italy	999.00

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:00:38	SELECT l.full_name AS learner_name, c.course_name	AS course_name, 8 row(s) returned	0.016 sec / 0.000 sec
2	22:01:26	SELECT l.full_name AS learner_name, c.course_name	AS course_name, 9 row(s) returned	0.000 sec / 0.000 sec
3	22:03:43	SELECT l.full_name AS learner_name, l.country AS country, FORMAT(SUM(p.quantity * c.unit_price), 2) AS total_spending	5 row(s) returned	0.016 sec / 0.000 sec

Object Info Session

Q2. Top 3 most purchased courses by quantity

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator: elearning_analysis

SCHEMAS

Filter objects

btmpty
btmpty1
chatbot_db
employee
flask_practice
form_data
leave_management_system
parkingmanagementsystem
plsql
practice
Tables
Views
Stored Procedures
Functions

Administration Schemas

No object selected

Information

Result Grid

Filter Rows: Limit to 1000 rows

Export: Wrap Cell Content: 1A Fetch rows: 10

135
136 -- Q2. Top 3 most purchased courses by quantity
137 SELECT
138 c.course_name AS course_name,
139 c.category AS category,
140 SUM(p.quantity) AS total_quantity_sold
141 FROM courses c
142 JOIN purchases p ON c.course_id = p.course_id
143 GROUP BY c.course_id, c.course_name, c.category
144 ORDER BY total_quantity_sold DESC, c.course_name ASC
145 LIMIT 3;
146

Result 5

course_name	category	total_quantity_sold
Python for Beginners	Beginner	4
Advanced Data Science	Data Science	3
Excel Fundamentals	Beginner	3

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:04:32	SELECT c.course_name AS course_name, c.category AS category, SUM(p.quantity) AS total_quantity_sold	3 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

Q3. Each category's total revenue and number of unique learners

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator: elearning_analysis

SCHEMAS

Filter objects

btmpty
btmpty1
chatbot_db
employee
flask_practice
form_data
leave_management_system
parkingmanagementsystem
plsql
practice
Tables
Views
Stored Procedures
Functions

Administration Schemas

Information

No object selected

Result Grid

Filter Rows: Limit to 1000 rows

Export: Wrap Cell Content: 1A

```

147 -- Q3. Each category's total revenue and number of unique learners
148 SELECT
149     c.category AS category,
150     FORMAT(SUM(p.quantity * c.unit_price), 2) AS total_revenue,
151     COUNT(DISTINCT p.learner_id) AS unique_learners
152 FROM courses c
153 JOIN purchases p ON c.course_id = p.course_id
154 GROUP BY c.category
155 ORDER BY SUM(p.quantity * c.unit_price) DESC;
156

```

category	total_revenue	unique_learners
Web Development	17,997.00	2
Data Science	14,997.00	2
Beginner	6,393.00	3
Marketing	4,998.00	1

Result 6

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:04:32	SELECT c.course_name AS course_name, c.category AS category, SUM(p.quantity) AS total_quantity	3 row(s) returned	0.000 sec / 0.000 sec
2	22:05:30	SELECT c.category AS category, FORMAT(SUM(p.quantity * c.unit_price), 2) AS total_revenue, COUNT(DISTINCT p.learner_id) AS unique_learners	4 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

Q4. Learners who purchased from more than one category

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator: elearning_analysis

SCHEMAS

Filter objects

btmpty
btmpty1
chatbot_db
employee
flask_practice
form_data
leave_management_system
parkingmanagementsystem
plsql
practice
Tables
Views
Stored Procedures
Functions

Administration Schemas

Information

No object selected

Result Grid

Filter Rows: Limit to 1000 rows

Export: Wrap Cell Content: 1A

```

156
157 -- Q4. Learners who purchased from more than one category
158 SELECT
159     l.full_name AS learner_name,
160     COUNT(DISTINCT c.category) AS categories_purchased
161 FROM learners l
162 JOIN purchases p ON l.learner_id = p.learner_id
163 JOIN courses c ON p.course_id = c.course_id
164 GROUP BY l.learner_id, l.full_name
165 HAVING COUNT(DISTINCT c.category) > 1
166 ORDER BY categories_purchased DESC;
167

```

learner_name	categories_purchased
Alice Johnson	2
Rahul Mehta	2
Kenji Sato	2

Result 7

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:04:32	SELECT c.course_name AS course_name, c.category AS category, SUM(p.quantity) AS total_quantity	3 row(s) returned	0.000 sec / 0.000 sec
2	22:05:30	SELECT c.category AS category, FORMAT(SUM(p.quantity * c.unit_price), 2) AS total_revenue, COUNT(DISTINCT p.learner_id) AS unique_learners	4 row(s) returned	0.000 sec / 0.000 sec
3	22:06:08	SELECT l.full_name AS learner_name, COUNT(DISTINCT c.category) AS categories_purchased FROM learners l JOIN purchases p ON l.learner_id = p.learner_id JOIN courses c ON p.course_id = c.course_id GROUP BY l.learner_id, l.full_name HAVING COUNT(DISTINCT c.category) > 1 ORDER BY categories_purchased DESC	3 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

Q5. Courses never purchased

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator: elearning_analysis

SCHMAS

Filter objects

- btmpty
- btmpty1
- chatbot_db
- employee
- flask_practice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- practise
- Tables
- Views
- Stored Procedures
- Functions

Administration Schemas

No object selected

167

168 -- Q5. Courses never purchased

169 • SELECT

170 c.course_name AS course_name,

171 c.category AS category

172 FROM courses c

173 LEFT JOIN purchases p ON c.course_id = p.course_id

174 WHERE p.purchase_id IS NULL;

175

Result Grid

course_name	category
UX Design Basics	Design

Result 8

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:07:17	SELECT c.course_name AS course_name, c.category AS category FROM courses c LEFT JOIN purchases p ON c...	1 row(s) returned	0.000 sec / 0.000 sec

SECTION 4: SUBQUERIES & CORRELATED SUBQUERIES (Q6-Q8)

Q6. Learners whose total spending is above the average learner spending

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator: elearning_analysis

SCHMAS

Filter objects

- btmpty
- btmpty1
- chatbot_db
- employee
- flask_practice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- practise
- Tables
- Views
- Stored Procedures
- Functions

Administration Schemas

No object selected

179

180 -- Q6. Learners whose total spending is above the average learner spending

181 SELECT

182 learner_name,

183 FORMAT(spending, 2) AS total_spending

184 FROM (

185 SELECT

186 l.learner_name AS learner_name,

187 SUM(p.amount) AS total_spending

188 FROM learners l

189 JOIN purchases p ON l.learner_id = p.learner_id

190 GROUP BY l.learner_id, l.learner_name

191) AS learner_totals

192 WHERE total_spending > (

193 SELECT AVG(learner_totals)

194 FROM (

195 SELECT SUM(spending) AS total_spending

196 FROM purchases p

197 JOIN learners l ON p.learner_id = l.learner_id

198 GROUP BY l.learner_id

199) AS avg_spending

200 ORDER BY total_spending DESC

201

202

Result Grid

learner_name	total_spending
Alice Johnson	8,996.00
Kenji Sato	16,997.00
Rahul Mehta	14,996.00

Result 9

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:09:14	SELECT learner name, FORMAT(total spending, 2) AS total spending FROM (SELECT l.full name	3 row(s) returned	0.032 sec / 0.000 sec

Q7. Courses whose price is higher than any course in the 'Beginner' category

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

btmpy
btmpy1
chatbot_db
employee
flask_practice
form_data
leave_management_system
parkingmanagementsystem
plsql
practise
Tables
Views
Stored Procedures
Functions
Administration
Schemas

Information

No object selected

203
204 -- Q7. Courses whose price is higher than any course in the 'Beginner' category
205 SELECT
206 course_name,
207 category,
208 FORMAT(unit_price, 2) AS unit_price
209 FROM courses
210 WHERE unit_price > ANY (
211 SELECT unit_price FROM courses WHERE category = 'Beginner'
212)
213 ORDER BY unit_price DESC;
214

Result Grid

course_name	category	unit_price
Python for Beginners	Beginner	999.00
Full Stack Web Development	Web Development	5,999.00
Advanced Data Science	Data Science	4,999.00
UX Design Basics	Design	3,499.00
Digital Marketing Basics	Marketing	2,499.00

Result 10

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:10:56	SELECT course_name, category, FORMAT(unit_price, 2) AS unit_price FROM courses WHERE unit_price > ANY (...	5 row(s) returned	0.016 sec / 0.000 sec

Q8. Learners who spent more than the average spending in their own country (correlated subquery)

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

btmpy
btmpy1
chatbot_db
employee
flask_practice
form_data
leave_management_system
parkingmanagementsystem
plsql
practise
Tables
Views
Stored Procedures
Functions
Administration
Schemas

Information

No object selected

215 -- Q8. Learners who spent more than the average spending in their own country (correlated subquery)
216 SELECT
217 l.full_name AS learner_name,
218 l.country AS country,
219 FORMAT(SUM(p.quantity * c.unit_price), 2) AS total_spending
220 FROM learners l
221 JOIN purchases p ON l.learner_id = p.learner_id
222 JOIN courses c ON p.course_id = c.course_id
223 GROUP BY l.learner_id, l.full_name, l.country
224 HAVING SUM(p.quantity * c.unit_price) > (
225 SELECT AVG(country_total_spending)
226 FROM (
227 SELECT l2.learner_id, l2.country AS learner_country, SUM(p2.quantity * c2.unit_price) AS spend
228 FROM learners l2
229 JOIN purchases p2 ON l2.learner_id = p2.learner_id
230 JOIN courses c2 ON p2.course_id = c2.course_id
231 GROUP BY l2.learner_id, l2.country
232) AS country_total
233 WHERE country_total.learner_country = l.country
234 ORDER BY total_spending DESC;
235

Result Grid

learner_name	country	total_spending
Alice Johnson	USA	8,996.00

Result 11

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:13:09	SELECT l.full_name AS learner_name, l.country AS country, FORMAT(SUM(p.quantity * c.unit_price), 2) AS total_spe...	1 row(s) returned	0.000 sec / 0.000 sec

SECTION 5: CTE, CASE, VIEW, AND NULL HANDLING (Q9-Q12)

Q9. CTE: total spending per learner, then filter learners spending above 10,000

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- btmpty
- btmpty1
- chatbot_db
- employee
- flask_practice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- practise
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

No object selected

Query Editor

```

227 -- SECTION 9: CTEs, CASE, VIEW, AND NULL HANDLING (ON Q12)
228
229 -- Q9. CTE: total spending per learner, then filter learners spending above 10,000
230
231 WITH learner_spending AS (
232   SELECT
233     l.learner_id,
234     l.full_name AS learner_name,
235     SUM(p.quantity * c.unit_price) AS total_spending
236   FROM learners l
237   JOIN purchases p ON l.learner_id = p.learner_id
238   JOIN courses c ON p.course_id = c.course_id
239   GROUP BY l.learner_id, l.full_name
240 )
241
242 SELECT
243   learner_name,
244   FORMAT(total_spending, 2) AS total_spending
245 FROM learner_spending
246 WHERE total_spending > 10000
247 ORDER BY total_spending DESC;

```

Result Grid

learner_name	total_spending
Kenji Sato	16,997.00
Rahul Mehta	14,996.00

Result 12

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:14:09	WITH learner_spending AS (SELECT learner_id, full_name AS learner_name, SUMlo.oua...	2 row(s) returned	0.000 sec / 0.000 sec

Q10. CASE: classify learners by spending tier

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- btmpty
- btmpty1
- chatbot_db
- employee
- flask_practice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- practise
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

No object selected

Query Editor

```

259 -- Q10. CASE: classify learners by spending tier
260
261 WITH learner_spending AS (
262   SELECT
263     learner_name,
264     FORMAT(total_spending, 2) AS total_spending,
265     CASE
266       WHEN total_spending > 15000 THEN 'High Value'
267       WHEN total_spending BETWEEN 8000 AND 15000 THEN 'Medium Value'
268       ELSE 'Low Value'
269     END AS spending_tier
270   FROM learner_spending
271   ORDER BY total_spending DESC;

```

Result Grid

learner_name	total_spending	spending_tier
Sophia Rossi	999.00	Low Value
Alice Johnson	8,996.00	Medium Value
Maria Garcia	2,397.00	Low Value
Kenji Sato	16,997.00	High Value
Rahul Mehta	14,996.00	Medium Value

Result 13

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:16:24	WITH learner_spending AS (SELECT l.full_name AS learner_name, SUMlo.ouantiv * c.unit price...	5 row(s) returned	0.016 sec / 0.000 sec

Q11. NULL Handling: all courses, replacing NULL purchase counts with 0

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: elearning_analysis x

SCHMAS

Filter objects

btmpty
btmpty1
chatbot_db
employee
flask_practice
form_data
leave_management_system
parkingmanagementsystem
plsql
practise
Tables
Views
Stored Procedures
Functions
Administration Schemas

No object selected

279
280 -- Q11. NULL Handling: all courses, replacing NULL purchase counts with 0
281 • SELECT
282 c.course_name AS course_name,
283 c.category AS category,
284 IFNULL(COUNT(p.purchase_id), 0) AS purchase_count,
285 COALESCE(SUM(p.quantity), 0) AS total_quantity_sold
286 FROM courses c
287 LEFT JOIN purchases p ON c.course_id = p.course_id
288 GROUP BY c.course_id, c.course_name, c.category
289 ORDER BY purchase_count DESC;
290

Result Grid

course_name	category	purchase_count	total_quantity_sold
Python for Beginners	Beginner	2	4
Advanced Data Science	Data Science	2	3
Full Stack Web Development	Web Development	2	3
Digital Marketing Basics	Marketing	1	2
Excel Fundamentals	Beginner	1	3

Result 15 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:17:29	SELECT c.course_name AS course_name, c.category AS category, IFNULL(COUNT(p.purchase_id), 0) AS purchase_count, COALESCE(SUM(p.quantity), 0) AS total_quantity_sold FROM courses c LEFT JOIN purchases p ON c.course_id = p.course_id GROUP BY c.course_id, c.course_name, c.category ORDER BY purchase_count DESC;	6 row(s) returned	0.000 sec / 0.000 sec

Q12. VIEW: category_performance_view

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: elearning_analysis x

SCHMAS

Filter objects

btmpty
btmpty1
chatbot_db
employee
flask_practice
form_data
leave_management_system
parkingmanagementsystem
plsql
practise
Tables
Views
Stored Procedures
Functions
Administration Schemas

No object selected

290
291 -- Q12. VIEW: category_performance_view
292 • CREATE OR REPLACE VIEW category_performance_view AS
293 SELECT
294 c.category AS category,
295 SUM(p.quantity * c.unit_price) AS total_revenue,
296 COUNT(p.purchase_id) AS number_of_purchases,
297 ROUND(SUM(p.quantity * c.unit_price) / COUNT(p.purchase_id), 2) AS avg_revenue_per_purchase
298 FROM courses c
299 JOIN purchases p ON c.course_id = p.course_id
300 GROUP BY c.category;
301

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:18:59	CREATE OR REPLACE VIEW category_performance_view AS SELECT c.category AS category, SUM(p.quantity * c.unit_price) AS total_revenue, COUNT(p.purchase_id) AS number_of_purchases, ROUND(SUM(p.quantity * c.unit_price) / COUNT(p.purchase_id), 2) AS avg_revenue_per_purchase FROM courses c JOIN purchases p ON c.course_id = p.course_id GROUP BY c.category;	AS c... 0 row(s) affected	0.016 sec

Query the view

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with various databases like 'btmpty', 'chatbot_db', and 'practic'. The main editor window shows a SQL query in the 'elearning_analysis' database. The query is as follows:

```

301
302 -- Query the view
303 SELECT
304     category,
305     FORMAT(total_revenue, 2) AS total_revenue,
306     number_of_purchases,
307     FORMAT(avg_revenue_per_purchase, 2) AS avg_revenue_per_purchase
308 FROM category_performance_view
309 ORDER BY total_revenue DESC;
310

```

Below the query editor, the 'Result Grid' shows the output of the query. It contains four columns: 'category', 'total_revenue', 'number_of_purchases', and 'avg_revenue_per_purchase'. The data is sorted by total revenue in descending order.

category	total_revenue	number_of_purchases	avg_revenue_per_purchase
Beginner	6,393.00	3	2,131.00
Marketing	4,998.00	1	4,998.00
Web Development	17,997.00	2	8,998.50
Data Science	14,997.00	2	7,498.50

At the bottom, the 'Action Output' pane shows the execution details: 1 row(s) returned, 0.000 sec / 0.000 sec.

5. Conclusion

The relational database was successfully designed, constrained, populated, and queried to meet every requirement of the capstone assignment. Key outcomes:

- A normalized 3-table schema was created with appropriate primary keys, foreign keys, data types, and NOT NULL constraints, ensuring referential integrity between learners, courses, and purchases.
- All three join types (INNER, LEFT, RIGHT) were used to combine the tables and present formatted, aliased, sorted output.
- Twelve analytical queries (Q1–Q12) were written and verified, covering aggregation, filtering, subqueries, a correlated subquery, a CTE, CASE-based classification, NULL handling, and a reusable view.
- The analysis revealed that Web Development and Data Science are the platform's highest-revenue categories, Python for Beginners is the most popular course by volume, and one course (UX Design Basics) generated no sales at all — a clear signal for further investigation or promotional action.

Overall, the project demonstrates a complete, working SQL workflow — from schema design through advanced analytical querying — and produces actionable business insights consistent with the platform management team's original objective.